

RideFlow Viva Keybook

Demo script, expected questions, and key concepts

RideFlow — Viva Keybook

Quick-reference guide for the project demo and viva. Everything below is phrased so it can be read aloud verbatim if needed.

1. Two-line elevator pitch

RideFlow is a relational-database-backed ride-hailing platform that lets riders book rides, drivers fulfil them, and admins monitor the fleet in real time. It demonstrates the full set of advanced database objects required by D3 — views, indexes, a stored procedure, three triggers, a scheduled event, and four DCL roles — on top of the D2 schema.

2. Project at a glance

Item	Value
Frontend	Vanilla HTML + CSS + JavaScript
Backend	Node.js 20, Express, <code>mysql2/promise</code>
Database	MySQL 8 (primary) and TiDB Cloud (bonus)
Auth	bcrypt password hashing, JWT (HS256)
Total tables	11 from D2 + 1 added in D3 (<code>admin_notifications</code>)
Total endpoints	32 across <code>/auth</code> , <code>/rider</code> , <code>/driver</code> , <code>/admin</code> , <code>/lookups</code>
Total dashboards	3 (rider, driver, admin)

3. End-to-end working flow (90-second demo script)

1. Open the home page; show the team footer at the bottom.
2. Click **Login** and sign in as `rider@demo.com` / `password`.
3. On the rider dashboard, choose pickup `Saddar, Karachi` and drop-off `DHA Phase 5, Karachi`, vehicle type **ECONOMY**, tick **Peak hour**, apply promo `WELCOME10`, click **Book**.
 - Expected response: fare 480, promo discount 48, you pay 432.
 - This proves `sp_calculate_fare` works ($480 = (100 + 25 \times 7 + 3 \times 15) \times 1.5$) and the promo discount is applied.
4. Log out → log in as `driver@demo.com` / `password`.
5. On the driver dashboard, click **Go online**; the new ride appears in the incoming queue.
6. Click **Accept** → **Start** → **Finish**.
 - Driver-side `Finish` only marks the payment as `PAID`. The fact that the ride moves to `COMPLETED` proves trigger 1 (`trg_payment_paid_complete_ride`) fired.
7. Log back in as the rider, give the driver a 1-star rating.
 - Driver's `avg_rating` drops below 3.5.
8. Log in as `admin@demo.com` / `password`.
 - **Alerts tab**: a fresh “Driver flagged for low rating” notification proves trigger 2 (`trg_driver_low_rating_flag`).
 - **Promo codes**: `WELCOME10`’s `usage_count` is now `2` — proves trigger 3 (`trg_promo_usage_increment`).
 - **Reports tab**: revenue per city, trips per driver, low-rated drivers (`HAVING < 3.5`), full-trip INNER JOIN, all-riders LEFT JOIN, per-rider history, drivers in city by rating, promo discount per ride.
 - **Dashboard**: KPIs and the two views (`ActiveRidesView`, `TopDriversView`) populate live.

4. Common viva questions and short answers

Schema and modelling

Q. Walk me through the schema. Eleven tables: `users`, `riders`, `drivers`, `vehicles`, `locations`, `fare_rules`, `rides`, `payments`, `promo_codes`, `ratings`, `support_tickets`. Each driver, rider, and admin is a `users` row plus a role-specific row (`drivers`, `riders`). The many-to-many between rides and promos is resolved through the `payments.promo_id` column. D3 adds one extra table `admin_notifications` and four columns (`promo_codes.usage_count`, `drivers.is_flagged`, `users.wallet_balance`, plus the timestamps already in D2).

Q. Why a separate `drivers` table — why not put fields on `users`? Class–subclass modelling. The shared identity attributes (name, email, phone, password hash, role) live on `users`; the role-

specific attributes (license number, avg_rating, is_flagged, avail_status) live on `drivers`. A driver row is created only when a user actually becomes a driver, and the foreign key keeps them in sync.

Q. Where are the constraints? `NOT NULL` and `DEFAULT` are on every business column. `UNIQUE` covers `users.email`, `drivers.license_number`, `vehicles.license_plate`, `promo_codes.code`. `CHECK` constraints enforce positive amounts and ratings between 1 and 5. Foreign keys all use `ON UPDATE CASCADE` and either `ON DELETE RESTRICT` (for hard references like a ride's driver) or `ON DELETE SET NULL` (for soft links like `admin_notifications.related_user_id`).

D3 SQL objects

Q. Why do you have two views? - `ActiveRidesView` joins `rides`, `users`, `drivers`, `vehicles`, and two copies of `locations` so the admin dashboard can render every in-flight ride with one query instead of six. - `TopDriversView` filters `drivers` to those with `avg_rating > 4.5`, used by the marketing/reports tab.

Q. How does `sp_calculate_fare` work? It looks up the ride's `rule_id`, multiplies `per_km_rate` × `distance_km`, adds `per_min_rate` × `duration_min` and `base_rate`, then multiplies by `surge_multiplier` if the caller passed `is_peak = TRUE`. The result is written to an OUT parameter so the caller (`backend/routes/rider.js`) can use it without a second round-trip.

Q. Why three triggers? 1. `trg_payment_paid_complete_ride` keeps the ride lifecycle truthful: the moment a payment is marked PAID, the corresponding ride is forced to `COMPLETED`. This means the front-end never has to do two updates and they cannot drift. 2. `trg_driver_low_rating_flag` enforces the rubric requirement that the admin be notified when a driver's average rating drops below 3.5. It both flips `drivers.is_flagged = 1` and inserts an `admin_notifications` row. 3. `trg_promo_usage_increment` makes `promo.usage_count` a true source of truth — it is updated atomically inside the same transaction that inserts the payment, so the `max_uses` check in the API can trust it.

Q. What does the event do? `ev_expire_promo_codes` runs every night at midnight and sets `is_active = 0` on any promo whose `valid_until` is in the past. This is what allows the application to keep using a single index-friendly `is_active = 1 AND valid_until >= CURDATE()` filter without a per-query date comparison.

Q. What does DCL look like in practice? Four roles. `driver_role` only has `SELECT` on `rides` (so a driver process can read jobs but not insert one). `rider_role` has `INSERT`, `SELECT` on `rides` and `payments`. `support_role` is granted read-only access — `DELETE` is explicitly revoked so a support agent cannot wipe rows. `admin_role` gets `ALL PRIVILEGES`.

TiDB Cloud / +5 bonus

Q. Why TiDB Cloud, and what changed? TiDB is a MySQL-compatible distributed SQL engine — the wire protocol matches, so the same `mysql2` driver works. We pointed `DB_TARGET=tidb` at a Starter cluster in AWS Oregon. TiDB does not implement stored procedures, triggers, or events, so the setup script skips `schema/04..07` for that target. The backend has fallback logic guarded by `if (target === 'tidb')` that reproduces the trigger and procedure behaviors at the application layer — fare math, `usage_count++`, `is_flagged` flagging, and ride-completion. Both targets pass the same end-to-end tests.

Q. Why keep the MySQL versions if the app does it inline? Because the rubric grades the SQL objects directly. We demo the procedure / triggers / event on local MySQL where they exist as real database objects; we run the bonus deploy on TiDB where they exist as application logic.

Backend

Q. How is auth done? On register or login, we hash the password with bcrypt (10 rounds) and sign a JWT containing `{ user_id, role }`. The token is stored in `localStorage` on the client and sent as `Authorization: Bearer ...` on every request. `middleware/auth.js` verifies the JWT and exposes `req.user`; `requireRole('ADMIN')` enforces RBAC.

Q. How do you avoid double-booking a driver? The booking endpoint picks a driver with a `NOT EXISTS` sub-query that filters out drivers who currently have a ride in `REQUESTED`, `ACCEPTED`, `DRIVER_EN_ROUTE`, or `IN_PROGRESS` state. The accept endpoint atomically flips `drivers.avail_status = 'ON_TRIP'` inside a transaction, so two riders cannot grab the same driver between the `SELECT` and the `UPDATE`.

Reporting queries

Q. Show me the eight required queries. They live in `backend/routes/admin.js` and the front-end calls them all from the **Reports** tab. - `SELECT` completed rides for a rider, ordered by date. - `SELECT` drivers in a city, ordered by rating. - `SUM(amount)` per city — `revenue-per-city`. - `COUNT(*)` per driver — `trips-per-driver`. - `AVG(score)` with `HAVING AVG(score) < 3.5` — `low-rated-drivers`. - `INNER JOIN` riders × rides × drivers × vehicles — `full-trip-report`. - `LEFT JOIN` so riders with zero rides still appear — `all-riders`. - `JOIN` on `payments` × `promo_codes` — `promo-discount`.

5. Numbers worth memorising

Item	Value
Fare formula	<code>(base + per_km × km + per_min × min) × surge</code>
Demo booking	<code>(100 + 25×7 + 3×15) × 1.5 = 480</code>
<code>WELCOME10</code> discount	10% → 48; you pay 432
Driver commission	20% (i.e. driver keeps 80%)
Low-rating threshold	<code>avg_rating < 3.5</code>
Top-driver threshold	<code>avg_rating > 4.5</code>
Demo password	<code>password</code> (for every seeded account)

6. Edge-case rebuttals (likely follow-ups)

- “What if a ride is force-cancelled mid-trip?” The `rides.ride_status` column is an `ENUM`, so an illegal value can’t even be inserted. The trigger only acts on `PAID` payments

and only bumps rides that are still in `IN_PROGRESS` or `ACCEPTED`, so a cancelled ride is never re-completed.

- **“What if two riders apply the same promo at the same time?”** `usage_count < max_uses` is checked inside the same transaction that inserts the payment, and the promo update is the last write before commit, so a race-conditioning rider would see the post-update count. In MySQL this is also enforced by the trigger; on TiDB by the inline `UPDATE promo_codes SET usage_count = usage_count + 1`.
- **“What if MySQL and TiDB drift?”** The setup script logs which target it is running against, applies only the files that target supports, and seeds identical data. The `/api/health` endpoint returns `{ db_target: "mysql" | "tidb" }` so the deployed environment is always self-identifying.
- **“Where is the ER diagram?”** `report/erd.png`, auto-generated from the schema (Mermaid source in `report/erd.mmd`).

7. One-line answers to “where is X?”

- **Where do ratings live?** `ratings` table. The trigger updates the driver's `avg_rating` after every insert.
- **Where is the wallet?** `users.wallet_balance` column, one endpoint pair (`GET /api/rider/wallet`, `POST /api/rider/wallet/topup`).
- **Where is the surge multiplier set?** `fare_rules.surge_multiplier`, one row per (city, vehicle type) pair.
- **Where is the schema versioned?** `schema/01..99` is the order of application; `setup-db.js` runs them in numeric order.
- **Where is the dev seed data?** `schema/99_seed.sql` — admin, three riders, three drivers, three locations, three vehicles, three promos, one fare rule per (city, type) pair, two pre-existing rides.